

23/10/2025

Đề tài: “Quản lý quyền truy cập động dùng cơ chế điều khiển truy cập Usage Control (UCON)”

Mục tiêu:

1. Tìm hiểu về UCON

2. Triển khai UCON

- Kiến trúc cơ bản của UCON
- Tìm hiểu thư viện có sẵn hỗ trợ UCON – Đánh giá
- Giải pháp mở rộng một hệ quản trị CSDL quan hệ để tích hợp UCON
- Triển khai cụ thể

3. Cài đặt thử nghiệm

4. Đánh giá, khuyến nghị

Gợi ý:

Usage control có phải là một cơ chế quản lý truy cập (access control)

ChatGPT said:

Rất chính xác —  **Usage Control (UCON) là một cơ chế quản lý truy cập (access control)**, nhưng **mở rộng hơn** các mô hình truy cập truyền thống như DAC, MAC, hay RBAC.

Giải thích chi tiết:

1. UCON là gì?

Usage Control (UCON) là mô hình do **Sandro Etalle và Ravi Sandhu** cùng cộng sự đề xuất đầu những năm 2000 (đặc biệt trong bộ mô hình **UCONABC**).

UCON được xem là **thế hệ tiếp theo của Access Control**, vì nó không chỉ quyết định **ai được phép truy cập tài nguyên**, mà còn **kiểm soát việc sử dụng (usage)** tài nguyên đó **trong suốt thời gian sử dụng**.

2. So sánh với các mô hình truy cập truyền thống

Tiêu chí	DAC / MAC / RBAC	UCON
Thời điểm kiểm soát	Trước khi truy cập (pre-access)	Trước, trong, và sau khi truy cập (pre, ongoing, post)
Kiểm tra điều kiện	Dựa trên quyền tĩnh (static permissions)	Dựa trên thuộc tính động (dynamic attributes)
Thu hồi quyền (revocation)	Thường không linh hoạt	Có thể thu hồi trong khi đang sử dụng
Mô hình chính sách	Quyền dựa vào chủ thể – đối tượng	Chính sách dựa vào thuộc tính và trạng thái
Ví dụ thực tế	File system, Role-based apps	Hệ thống IoT, đám mây, e-health, học trực tuyến,...

3. Ba thành phần cơ bản của UCON (UCONABC)

- **A – Authorizations:** Kiểm tra quyền trước hoặc trong khi sử dụng tài nguyên.
- **B – Obligations:** Yêu cầu người dùng thực hiện hành động nào đó để duy trì quyền (ví dụ: xác thực định kỳ).
- **C – Conditions:** Các điều kiện của môi trường (ví dụ: thiết bị đang online, dữ liệu chưa quá cũ...).

UCON còn có các khái niệm:

- **Mutable attributes:** Thuộc tính có thể thay đổi trong quá trình truy cập (ví dụ: thời lượng sử dụng, số lượt truy cập,...).
- **Continuity of control:** Kiểm soát **liên tục** thay vì chỉ tại thời điểm bắt đầu.

4. Kết luận

➡ **UCON** là một mô hình quản lý truy cập, nhưng **vượt trội hơn Access Control truyền thống** vì:

- Kết hợp cả **authorization, obligation, condition**.
- Hỗ trợ **kiểm soát liên tục và động**.
- Phù hợp với **IoT, Cloud, Streaming, eHealth, eLearning**, nơi quyền truy cập thay đổi theo **thời gian, ngữ cảnh, và trạng thái thiết bị/người dùng**.

Nếu bạn muốn, tôi có thể giúp bạn **vẽ sơ đồ so sánh trực quan** giữa **Access Control truyền thống (DAC/MAC/RBAC)** và **UCON** để thấy rõ mức độ mở rộng và khác biệt. Bạn có muốn tôi tạo sơ đồ đó không?

You said:

Hiện tại có thư viện cài đặt UCON để quản lý chính sách truy cập động không?

ChatGPT said:

Có — hiện có một số thư viện hỗ trợ hoặc mở rộng để triển khai mô hình Usage Control (UCON) (Usage Control) — tức là cơ chế quản lý truy cập động với các yếu tố Pre/Ongoing/Post, thuộc tính động, obligation/condition. Dưới đây là chi tiết và lưu ý khi sử dụng.

✅ Các thư viện có sẵn

Một thư viện tiêu biểu là:

- Casbin (một thư viện kiểm soát truy cập nổi tiếng) cung cấp phần mở rộng gọi là **casbin-ucon** để hỗ trợ mô hình UCON. [casbin.org+1](https://casbin.org/)

Ví dụ sử dụng:

```
go get github.com/casbin/casbin-ucon
import (
    "github.com/casbin/casbin/v2"
    "github.com/casbin/casbin-ucon"
)
```

- Thư viện hỗ trợ thêm **Conditions** và **Obligations**, và cho phép **continuous monitoring / session-based control** theo mô hình UCON. [casbin.org+1](https://casbin.org/)
 - Ngoài ra có các nghiên cứu và kiến trúc mở đã triển khai UCON trong môi trường dữ liệu/nền tảng (ví dụ: sử dụng FIWARE cho việc chia sẻ dữ liệu công nghiệp). [MDPI](#)
-

Lưu ý và hạn chế khi sử dụng

Mặc dù có thư viện hỗ trợ nhưng khi bạn triển khai thực tế vẫn cần lưu ý:

- Thư viện như **casbin-ucon** là mở rộng và không “đầy đủ” toàn bộ tính năng lý thuyết UCON (ví dụ ongoing revocation, mutability attribute phức tạp) — bạn có thể cần **bổ sung** logic riêng.
 - Việc triển khai **monitoring liên tục** (ongoing control) và **thu hồi quyền trong khi phiên đang diễn ra** vẫn đòi hỏi kiến trúc bên ngoài (session tracking, timer, trigger) — thư viện chỉ hỗ trợ một phần.
 - Phần **Post-action / Attribute update** (mutability) thường bạn phải tự triển khai phần cập nhật thuộc tính người dùng/đối tượng sau truy cập. Thư viện chỉ hỗ trợ framework chính sách.
 - Hiệu suất: kiểm soát ongoing, monitoring có thể ảnh hưởng đến hiệu năng; bạn cần đánh giá lại khi áp dụng thư viện vào hệ thống lớn.
 - Tích hợp: nếu hệ thống của bạn là RDBMS/trigger/loT như đề tài của bạn, bạn sẽ cần **kết nối thư viện kiểm soát truy cập (như Casbin) với cơ chế trigger/procedure hoặc middleware** để hoàn thiện UCON.
-

Kết luận cho đề tài của bạn

Vì đề tài của bạn là “**Triển khai cơ chế UCON trên hệ quản trị cơ sở dữ liệu ... để quản lý quyền truy cập động**”, nên bạn có thể:

- Chọn sử dụng thư viện như casbin-ucon để **quản lý chính sách và quyết định PDP**.
- Sau đó, tích hợp với RDBMS (PostgreSQL/Oracle) hoặc Trigger/Procedure để xử lý **PEP và thuộc tính động (PIP)**.
- Nếu thư viện không đủ tính năng ongoing revocation hoặc session control, bạn có thể **mở rộng thêm** bằng trigger + cron job + bảng log để cập nhật.

Với cách này, bạn vừa tận dụng thư viện đã có để tiết kiệm thời gian, đồng thời vẫn triển khai được phần chuyên sâu (ongoing, revocation) mà thư viện có thể chưa hỗ trợ đầy đủ — rất phù hợp cho một khóa luận tốt nghiệp.

Nếu bạn muốn, tôi có thể tìm giúp bạn thêm **3-5 thư viện/khung khác hỗ trợ UCON hoặc usage control**, kèm theo **so sánh ưu/nhược** để bạn chọn cho đề tài. Bạn muốn tôi tìm không?

You said:

Cho tôi 3-5 thư viện hỗ trợ UCON và đánh giá giúp tôi mức độ đáp ứng mục tiêu điều khiển truy cập động?

ChatGPT said:

Dưới đây là **3 thư viện/khung mã nguồn mở** hỗ trợ hoặc liên quan tới mô hình Usage Control (UCON), cùng với đánh giá mức độ đáp ứng mục tiêu “điều khiển truy cập động (ongoing control, mutability, obligations/conditions)”. Bạn có thể dùng phần này để lựa chọn thư viện phù hợp cho đề tài hoặc làm luận cứ trong phần “Công cụ triển khai”.

STT	Thư viện / Khung	Ngôn ngữ / nền	Tính năng hỗ trợ UCON	Đánh giá mức độ đáp ứng truy cập động
1	casbin-ucan (extension của Casbin) casbin.org+2GitHub+2	Go (và có thể dùng thông qua wrapper)	Hỗ trợ session-based, điều kiện (conditions), obligation, monitoring liên tục. Ví dụ: <code>CreateSession</code> , <code>StartMonitoring(sessionID)</code> trong README. GitHub	Đánh giá: Cao về hỗ trợ phần “ongoing” hơn nhiều thư viện khác; nhưng vẫn ở cấp “extension” và cần nhiều phần mở rộng thủ công (ví dụ cập nhật thuộc tính động, kết nối với trigger DB) → phù hợp “một phần lớn” mục tiêu truy cập động.

STT	Thư viện / Khung	Ngôn ngữ / nền	Tính năng hỗ trợ UCON	Đánh giá mức độ đáp ứng truy cập động
2	Framework "Usage Control service" tích hợp vào Cloud IaaS (theo bài báo) webhost.services.iit.cnr.it	Java (trong bài báo)	Hỗ trợ UCON đầy đủ: pre-, ongoing-, post- quyết định; revocation khi attribute thay đổi; trong môi trường cloud.	Đánh giá: Rất tốt về khái niệm và lý thuyết; nhưng nếu tìm thư viện mã nguồn mở để sử dụng ngay thì có thể phải triển khai lại hoặc chuyển đổi; ít "plug & play". Đáp ứng mục tiêu "điều khiển truy cập động" tốt, nhưng chi phí tích hợp cao hơn.
3	Khung kéo theo DSL và mô hình cho UCON (ví dụ DSL to define UCON security policies) Modeling Languages	Eclipse/Java modelling tool	Hỗ trợ mô hình hóa policy UCON (Subjects, Objects, Rights, Rules, Environment) với khả năng biểu diễn pre/on/post, thuộc tính mutable.	Đánh giá: Hữu ích cho phần thiết kế và thực hiện chính sách nhưng không phải thư viện thực thi truy cập động ngay lập tức ; bạn vẫn phải triển khai engine enforcement. Đáp ứng phần "thiết kế" tốt nhưng phần "thực thi ongoing control" cần bổ sung.

- Nếu bạn muốn **triển khai nhanh** phần “quản lý chính sách + quyết định truy cập động” và tập trung vào phần database/trigger của đề tài, thì **casbin-ucan** là lựa chọn khả thi nhất vì có thư viện sẵn và hỗ trợ khá tốt cho UCON.
- Nếu bạn muốn **ngiên cứu sâu hơn** về UCON, đặc biệt là ongoing revocation/attribute mutability/monitoring, bạn có thể sử dụng framework số 2 làm tham khảo và tích hợp thủ công một phần vào hệ thống của bạn (ví dụ viết module UCON service riêng).
- Nếu bạn tập trung vào phần **mô hình hóa chính sách và kiến trúc** hơn là triển khai hoàn chỉnh, thì khung số 3 giúp bạn chứng minh “thiết kế chính sách UCON” bài bản, rồi bạn có phần thực thi riêng.

✅ Kết luận cho đề tài của bạn

Vì đề tài của bạn là “**Triển khai cơ chế UCON trên RDBMS để quản lý quyền truy cập động**”, nên:

- Bạn có thể chọn **casbin-ucan** làm **khung thư viện chính để xử lý phần chính sách và quyết định (PDP, PEP logic)**.
- Sau đó, **kết nối với RDBMS (PostgreSQL/Oracle)** để thực hiện phần **bảng thuộc tính động (PIP)**, phần **trigger/procedure kiểm tra ongoing condition** và phần **thu hồi quyền trong phiên**.
- Nếu casbin-ucan không hỗ trợ đầy đủ phần “thu hồi trong phiên hay monitoring liên tục” bạn hoàn toàn có thể **mở rộng bằng trigger/time-job** trong RDBMS — điều này vẫn hợp với mục tiêu “quản lý quyền truy cập động”.

Nếu bạn muốn, tôi có thể tìm thêm **2 thư viện khác** (ngoài 3 trên) — có thể ít phổ biến hơn — và **so sánh chi tiết theo các tiêu chí: ongoing control, attribute mutability, ease of integration với RDBMS** để bạn có bảng chọn thư viện phù hợp nhất. Bạn có muốn tôi tìm hai cái đó không?

Với đề tài khóa luận tốt nghiệp “Triển khai cơ chế Usage Control (UCON) trên hệ quản trị cơ sở dữ liệu PostgreSQL/Oracle để quản lý quyền truy cập động”, với nội dung như sau: Mục tiêu: Xây dựng module mở rộng hỗ trợ chính sách UCON trên một RDBMS phổ biến, cho phép cập nhật quyền trong khi phiên làm việc đang diễn ra. Nội dung chính: Phân tích yêu cầu và kiến trúc UCON. Thiết kế bảng chính sách và bộ máy thực thi (Policy Decision Point & Policy Enforcement Point). Viết trigger/procedure để xử lý ongoing conditions và revoke/update quyền khi vi phạm. Kiểm thử trên dữ liệu mẫu.” Hãy làm rõ thêm giúp tôi ở các nội dung chính và cho tôi một số ví dụ phù hợp dùng UCON.

ChatGPT said:

Dưới đây là phần mở rộng, phân tích rõ hơn cho các **nội dung chính** của đề tài cùng với **ví dụ điển hình phù hợp với mô hình UCON** trong môi trường cơ sở dữ liệu:

✔ **Làm rõ các nội dung chính**

1. Phân tích yêu cầu và kiến trúc UCON

- **Xác định đặc trưng của UCON so với DAC, MAC, RBAC:**
 - **Thuộc tính động (mutable attributes)** của user, resource, environment.
 - **Quyền có thể bị cập nhật hoặc thu hồi trong khi phiên truy cập đang diễn ra (ongoing control).**
 - **Quyết định truy cập không chỉ dựa trên yêu cầu ban đầu mà liên tục được đánh giá lại (continuous evaluation).**
- **Xác định điểm can thiệp hợp lý vào PostgreSQL/Oracle để:**
 - Ghi nhận hành vi truy cập (access events logging).
 - Kích hoạt kiểm tra chính sách tại thời điểm pre, ongoing, và post theo chuẩn UCONabc.
- **Kiến trúc tích hợp UCON vào RDBMS:**
 - **PDP (Policy Decision Point):** bảng chính sách, thủ tục đánh giá.
 - **PEP (Policy Enforcement Point):** trigger/permission hooks tự động kích hoạt khi người dùng SELECT/UPDATE/DELETE/INSERT.

2. Thiết kế bảng chính sách và bộ máy thực thi PDP/PEP

- **Bảng chính sách (UCON_POLICY):**

Policy_ID	Subject_Attr	Object_Attr	Condition	Obligation	Effect (Permit/Deny/Revoke)
-----------	--------------	-------------	-----------	------------	-----------------------------

- **Bảng thuộc tính (User_Attributes, Object_Attributes):**
 - Lưu các thuộc tính có thể thay đổi: Access_Count, Last_Access_Time, Trust_Score, Session_Duration, Data_Sensitivity_Level.
- **PDP** là các hàm/logic đánh giá điều kiện, ví dụ:
- `FUNCTION evaluate_policy(user, object, action) RETURNS BOOLEAN;`
- **PEP** là Trigger hoặc Policy Hook:
 - Trước khi thực thi truy vấn (BEFORE SELECT/UPDATE)
 - Hoặc định kỳ kiểm tra trong phiên truy cập (ON UPDATE SESSION_STATS)

3. Trigger/Procedure xử lý ongoing conditions và Revoke/Update quyền

Ví dụ: Người dùng chỉ được xem bảng Patients nếu **Trust_Score $\geq$ 70**, trong quá trình truy cập nếu **Access_Count vượt 100 yêu cầu**, thì:

- Trigger kiểm tra trust score mỗi lần truy cập.
- Khi Access_Count > 100, trigger tự động:
- REVOKE SELECT ON Patients FROM userX;
- INSERT INTO Audit_Log(...);
- Nếu thuộc tính thay đổi (ví dụ Trust_Score giảm do vi phạm), hệ thống tự cập nhật quyền ngay lập tức.

4. Kiểm thử trên dữ liệu mẫu

- Tạo **3 nhóm người dùng**: Doctor, Nurse, Intern.
- Tạo bảng Medical_Record với thuộc tính Sensitivity_Level.
- Thiết lập chính sách:
 - Doctor xem toàn bộ.
 - Nurse chỉ xem khi Session_Time < 30 phút.
 - Intern chỉ xem dữ liệu đã được ẩn thông tin cá nhân (de-identified).
- Thực hiện truy cập liên tục để mô phỏng **ongoing constraint** và ghi nhận log.

 Một số tình huống thực tế phù hợp với mô hình UCON

Ngữ cảnh	Loại thuộc tính UCON	Điều kiện Ongoing	Hành vi hệ thống
Bác sĩ xem hồ sơ bệnh nhân	SessionDuration, AccessFrequency	Nếu quá 30 phút liên tục → cảnh báo & yêu cầu xác thực lại	Tự động RE-AUTH hoặc tạm khóa
Nhân viên nội trú xem dữ liệu	TrustScore	Nếu TrustScore < ngưỡng → thu hồi quyền ngay cả khi đang truy cập	REVOKE SELECT trên session
Truy cập dữ liệu nhạy cảm	DataSensitivity, UserRank	Nếu người dùng cố tải nhiều bản ghi trong thời gian ngắn → giới hạn tốc độ (Rate Limiting)	Trigger SLEEP hoặc giới hạn query
Tài liệu chia sẻ theo thời	Valid_Until	Khi NOW() > Valid_Until → tự ngắt quyền xem trong phiên	Trigger LOGOUT hoặc DENY

Ngữ cảnh	Loại thuộc tính UCON	Điều kiện Ongoing	Hành vi hệ thống
gian			ONGOING

 Gợi ý mở rộng nhằm tăng tính học thuật của đề tài

- Đề xuất cách **mã hóa chính sách dưới dạng JSON hoặc XACML-like** để hệ thống dễ mở rộng.
- So sánh hiệu năng giữa:
 - **Chỉ kiểm tra Pre-access** (như RBAC truyền thống)
 - **Kiểm tra Ongoing (UCON)** khi số lượng session tăng.
- Tích hợp **AUDIT LOG + MACHINE LEARNING** để cập nhật TrustScore động.

 Nếu bạn muốn, tôi có thể tiếp tục giúp:

- Thiết kế **bảng chi tiết** chính sách theo đúng format triển khai.
- Viết **mẫu Trigger/Procedure** cho PostgreSQL và Oracle.
- Xây dựng **kịch bản kiểm thử và script sinh dữ liệu**.
- Viết mục **"Ý nghĩa học thuật và thực tiễn"** cho báo cáo khóa luận.

Bạn muốn làm bước nào tiếp trước?

You said:

Cho tôi ngữ cảnh minh họa UCON làm rõ đặc trưng của UCON tiếp theo câu hỏi trước. Hướng dẫn rõ cách thể hiện kiến trúc tích hợp UCON vào RDBMS ở PDP và PEP.

ChatGPT said:

Dưới đây là **ngữ cảnh minh họa rõ ràng** và **hướng dẫn cách thể hiện kiến trúc PDP/PEP của UCON trong hệ quản trị CSDL quan hệ** như PostgreSQL/Oracle — có thể đưa trực tiếp vào phần **"Mô hình và kiến trúc đề xuất"** của khóa luận.

 Ngữ cảnh minh họa: Quản lý truy cập hồ sơ bệnh nhân theo mô hình UCON

Tình huống thực tế

- Bác sĩ **Dr. Anna** đăng nhập vào hệ thống để xem bảng **Medical_Record**.
- Điều kiện truy cập quy định:
 1. **Pre-access:** Dr. Anna phải thuộc nhóm Doctor và có Trust_Score ≥ 70 .

2. **Ongoing Control:** Trong quá trình truy cập:
- Nếu thời gian phiên > **30 phút liên tục**, hệ thống **yêu cầu xác thực lại**.
 - Nếu Access_Frequency > **50 bản ghi trong 5 phút**, hệ thống **tự động tạm khóa quyền SELECT ngay khi truy cập đang diễn ra**.
3. **Post-control:** Hệ thống ghi lại Audit_Log và nếu vi phạm nhiều lần → **giảm Trust_Score xuống**, ảnh hưởng đến quyền truy cập ở phiên sau.

👉 **Điểm thể hiện đặc trưng UCON:**

Cơ chế	Quyết định quyền	Thời điểm đánh giá	Đặc trưng
DAC/RBAC truyền thống	Quyền cấp một lần → giữ cố định trong suốt phiên	Chỉ đánh giá lúc bắt đầu truy cập	Không thay đổi quyền khi đang truy cập
UCON (Usage Control)	Quyền được cấp, thu hồi hoặc thay đổi trong lúc truy cập	Liên tục đánh giá (Pre, Ongoing, Post)	Quyền động — Mutable Attributes + Continuous Enforcement

🏗️ **Kiến trúc tích hợp UCON vào PostgreSQL/Oracle**

Thành phần hệ thống

Thành phần	Vai trò trong UCON	Triển khai trên RDBMS
Policy Decision Point (PDP)	Đánh giá chính sách dựa trên thuộc tính	Thủ tục (FUNCTION / PACKAGE) evaluate_policy() đọc bảng UCON_POLICY
Policy Enforcement Point (PEP)	Chặn/cho phép các hành động DB theo quyết định PDP	Trigger BEFORE SELECT/UPDATE/DELETE hoặc LOGON TRIGGER + SESSION EVENT TRIGGER
Policy Information Point (PIP)	Nơi lưu thông tin thuộc tính động	Bảng USER_ATTRIBUTES, SESSION_STATS, DATA_ATTRIBUTES
Audit / Feedback Mechanism	Ghi nhận sự kiện → cập nhật thuộc tính	Trigger AFTER ACCESS để cập nhật Trust_Score, Access_Count

🌀 **Cách biểu diễn kiến trúc trong báo cáo (gợi ý vẽ sơ đồ)**

[Client/User Session]

|

v

| PostgreSQL / Oracle |

||

| 1. User Access Request (e.g., SELECT Medical_Record) |

|||

| v |

| _____ | _____ |

|| PEP | --Call--> | PDP ||

|| Trigger || evaluate_policy() ||

| _____ | _____ |

|| Reads Attributes |

|| _____ |

| L-----> | PIP (Attributes DB) | |

| _____ |

||

| If Deny → REVOKE / ABORT Query (Immediate Enforce) |

| If Allow → Execute Query Then Log + Update Attributes |

✅ **Hình trên thể hiện rõ:**

- **PEP = Trigger** (nơi can thiệp thực tế)

- **PDP = FUNCTION** đánh giá chính sách
 - **PIP = bảng thuộc tính động**
 - **Decision Loop liên tục** → thể hiện tinh thần "Ongoing Control" của UCON
-

Cách bạn có thể viết vào báo cáo (mẫu diễn giải)

"Trong kiến trúc tích hợp, trigger trong PostgreSQL/Oracle đóng vai trò như Policy Enforcement Point (PEP), chịu trách nhiệm chặn mọi yêu cầu truy cập và chuyển thông tin này đến PDP để đánh giá. PDP được triển khai dưới dạng thủ tục evaluate_policy() truy vấn bảng chính sách và bảng thuộc tính (PIP). Nếu vi phạm, trigger lập tức thu hồi quyền hoặc hủy truy vấn ngay cả khi truy cập đang diễn ra, thể hiện rõ đặc trưng Ongoing Control của UCON."

Bước tiếp theo bạn muốn?

Tôi có thể tiếp tục với một trong các nội dung:

1. **Thiết kế bảng chính sách và bảng thuộc tính chi tiết (SQL script)**
2. **Mã mẫu Trigger + Function PDP/PEP chuẩn UCON**
3. **Sơ đồ kiến trúc dạng đẹp có thể chèn vào báo cáo**
4. **Viết đoạn "Ý nghĩa học thuật và thực tiễn của việc tích hợp UCON vào RDBMS"**

Bạn chọn phần nào để tôi triển khai tiếp?

You said:

Cho tôi thêm 10 chính sách UCON trong ngữ cảnh y tế như ví dụ trước.

ChatGPT said:

Dưới đây là **10 chính sách UCON điển hình trong ngữ cảnh quản lý truy cập dữ liệu y tế**, thể hiện rõ **Pre-Authorization, Ongoing Control**, và **Post-Update Attribute** — có thể ghi trực tiếp vào bảng chính sách trong khóa luận.

Danh sách 10 chính sách UCON cụ thể

Mã chính sách	Ngữ cảnh	Pre-Condition (Trước truy cập)	Ongoing Control (Trong khi truy cập)	Post-Action / Attribute Update
UCON-P1	Bác sĩ xem hồ sơ bệnh nhân	Vai trò = Doctor AND Trust_Score $\geq$ 70	Nếu Session_Time > 30 phút → Yêu cầu xác thực lại	Ghi log + +1 Access_Count
UCON-P2	Điều dưỡng xem hồ sơ	Vai trò = Nurse AND Sensitivity_Level $\leq$ 2	Nếu Access_Frequency > 50 records / 5 phút → Tạm khóa quyền SELECT	Giảm Trust_Score -5 nếu vi phạm
UCON-P3	Nội trú (Intern) truy cập	Vai trò = Intern AND Data_Masked = TRUE	Nếu Intern cố truy vấn thông tin nhận dạng (Name, Address) → Chặn truy vấn ngay lập tức	Ghi Audit_Log, flag Suspicious_Access
UCON-P4	Chia sẻ hồ sơ tạm thời	Nếu User được cấp quyền tạm (Valid_Until > NOW)	Khi NOW > Valid_Until → REVOKE quyền ngay khi đang mở session	Log "Expired Access"
UCON-P5	Tải xuống dữ liệu hàng loạt	Vai trò $\in$ {Doctor, Researcher} AND Export_Permission = TRUE	Nếu số bản ghi tải > 2000 → Hủy query ngay lập tức	Thiết lập Rate_Limited = TRUE
UCON-P6	Bệnh nhân tự xem hồ sơ (Patient Portal)	User_ID = Patient_ID và MFA_Verified = TRUE	Nếu Failed_Attempts > 3 trong phiên → Logout cưỡng bức	Tăng giá trị Failed_Attempts_Log
UCON-P7	Nhà nghiên cứu phân tích dữ liệu	Vai trò = Researcher AND Dataset = De-identified	Nếu truy vấn cột chứa PII (Thông tin cá nhân) → Chặn view và cảnh báo	Ghi vào bảng Policy_Violation
UCON-P8	Kiểm soát thao tác UPDATE hồ sơ	Chỉ cho phép UPDATE nếu	Nếu thời gian truy cập chỉnh sửa > 10 phút →	Log Editing_Session_Time

Mã chính sách	Ngữ cảnh	Pre-Condition (Trước truy cập)	Ongoing Control (Trong khi truy cập)	Post-Action / Attribute Update
		User_Role ∈ {Doctor, Admin}	Tự động lưu tạm, yêu cầu xác nhận tiếp tục	
UCON-P9	Cấp quyền khẩn cấp (Emergency Access – Break Glass Mode)	Vai trò = Doctor AND Emergency_Flag = TRUE	Giới hạn thời gian truy cập khẩn cấp tối đa 15 phút	Sau khi kết thúc, tự động revoke quyền khẩn cấp
UCON-P10	Học viên quan sát (Observer Mode)	Vai trò = Student_Observer (chỉ SELECT hạn chế)	Nếu query chứa thông tin điều trị chi tiết → Tự động che khuất (mask) dữ liệu	Cập nhật Attribute Has_Seen_Data = TRUE

 Gợi ý sử dụng trong báo cáo

Bạn có thể trình bày các chính sách trên bằng cách:

1. **Lập bảng UCON_POLICY:**
2. Policy_ID | Subject_Attr | Object_Attr | Condition_Type (Pre/Ongoing/Post) | Condition | Effect
3. **Nhấn mạnh sự khác biệt so với RBAC:**
 - Với RBAC, quyền chỉ kiểm tra một lần.
 - Với UCON, các chính sách như P2, P4, P6, P9 thể hiện rõ **thu hồi quyền ngay trong phiên** → **đặc trưng UCON**.
4. **Chọn 2-3 chính sách để triển khai trigger thực tế.**

 Nếu bạn muốn, tôi có thể tiếp tục với:

-  **Sinh bảng SQL mẫu cho 10 chính sách trên** (INSERT INTO UCON_POLICY ...)
-  **Tạo mẫu bảng thuộc tính động (Trust_Score, Access_Count, Session_Time, ...)**
-  **Sinh Trigger / Function giả lập PDP và PEP cho một chính sách cụ thể**

 **Bạn muốn tôi triển khai theo hướng SQL triển khai thực tế hay mô tả kiến trúc dạng sơ đồ trước?**

You said:

UCON cũng là một mô hình điều khiển truy cập dựa trên thuộc tính, tôi có cần phải dùng một thư viện hỗ trợ ABAC để triển khai UCON hay là tôi chỉ cần mô phỏng UCON trong một hệ quản trị RDBMS là được?

ChatGPT said:

Bạn **không bắt buộc phải dùng thư viện ABAC** để triển khai UCON. Trong **bối cảnh khóa luận và mục tiêu "tích hợp UCON vào RDBMS"**, bạn **hoàn toàn có thể mô phỏng UCON trực tiếp trong PostgreSQL/Oracle** thông qua:

- **Bảng chính sách** (policy table)
- **Bảng thuộc tính động của user, object và session**
- **Trigger / Function đóng vai trò PDP và PEP**
- **Cơ chế ghi nhận sự kiện (audit) để cập nhật thuộc tính sau truy cập — đặc trưng Post-Update của UCON**

 Diễn giải học thuật để đưa vào báo cáo

"Mặc dù UCON thuộc họ mô hình điều khiển truy cập dựa trên thuộc tính (ABAC-based), nhưng UCON có đặc trưng riêng là khả năng quản lý thuộc tính **động** và kiểm soát quyền **liên tục trong suốt phiên truy cập**, khác với ABAC truyền thống chỉ đánh giá tại thời điểm bắt đầu truy cập. Do đó, việc triển khai UCON **không bắt buộc phải sử dụng thư viện ABAC**, mà có thể được mô phỏng trực tiếp trong hệ quản trị cơ sở dữ liệu thông qua việc xây dựng bảng thuộc tính động và trigger xử lý ongoing control."

 So sánh để làm rõ luận điểm trong khóa luận

Tiêu chí	ABAC truyền thống (như XACML)	UCON mô phỏng trên RDBMS
Thời điểm đánh giá	Chỉ Pre-Authorization	Pre + Ongoing + Post
Thuộc tính	Tĩnh (static attributes)	Mutable attributes (TrustScore, SessionTime...)
Cơ chế thực thi	Thông qua Policy Engine ngoài DB	Trigger/Procedure trực tiếp trong DB
Công cụ triển khai	Cần thư viện như WSO2, AuthZForce, OPA, Keycloak...	Không cần thư viện ngoài – dùng SQL/PLpgSQL/PLSQL

Tiêu chí	ABAC truyền thống (như XACML)	UCON mô phỏng trên RDBMS
Kiểm soát động	Không giám sát trong phiên	Có thể REVOKE quyền ngay khi truy cập đang diễn ra

🚩 Kết luận để đưa vào phần phạm vi triển khai

"Trong phạm vi khóa luận này, để đảm bảo khả năng tích hợp sâu vào hệ quản trị CSDL PostgreSQL/Oracle mà không phụ thuộc vào thư viện ngoài, UCON sẽ được **mô phỏng trực tiếp bằng bảng chính sách và trigger database nội bộ**. Cách tiếp cận này vừa thể hiện được bản chất UCON, vừa có tính thực tế cao và dễ kiểm thử."

🎓 Gợi ý hướng mở rộng (nếu muốn làm thêm cho phần "Nghiên cứu phát triển")

- Nếu muốn thể hiện **tính nghiên cứu sâu hơn**, bạn có thể nói:

"Trong tương lai, có thể tích hợp thêm tầng Policy Engine tuân theo chuẩn XACML hoặc kết nối với Open Policy Agent (OPA) để hỗ trợ giải thích chính sách (policy explanation) và quản lý chính sách dưới dạng JSON hoặc XACML."

👉 **Bạn có muốn tôi đề xuất cấu trúc bảng SQL mô phỏng UCON với thuộc tính động + trigger mẫu cho một chính sách cụ thể không?** Chúng ta có thể bắt đầu từ **UCON-P1 (Session timeout 30 phút)** hoặc **UCON-P2 (Rate limiting truy vấn)**.

You said:

đề xuất cấu trúc bảng SQL mô phỏng UCON với thuộc tính động + trigger mẫu cho một chính sách cụ thể gồm UCON-P1 và UCON-P2

ChatGPT said:

Dưới đây là **thiết kế bảng + trigger mẫu triển khai UCON-P1 và UCON-P2 trong PostgreSQL hoặc Oracle**, theo đúng kiến trúc **PDP + PEP + PIP**.

🎯 Chính sách chọn để triển khai

Mã chính sách	Mô tả
UCON-P1	Bác sĩ (Doctor) được phép SELECT Medical_Record , nhưng nếu Session_Time > 30 phút , hệ thống yêu cầu xác thực lại hoặc chặn truy cập tiếp .
UCON-P2	Nếu một user truy cập > 50 bản ghi trong vòng 5 phút , hệ thống tự động thu hồi quyền SELECT tạm thời (Rate limiting) và ghi log vi phạm.

1. Bảng thuộc tính người dùng (PIP – Policy Information Point)

-- Bảng thuộc tính động của user

```
CREATE TABLE USER_ATTRIBUTES (

    user\_id VARCHAR2(50) PRIMARY KEY,

    role VARCHAR2(50),

    trust\_score INT DEFAULT 100,

    access\_count INT DEFAULT 0,           \-- Số lần truy cập gần nhất

    last\_access\_time TIMESTAMP,

    is\_rate\_limited BOOLEAN DEFAULT FALSE

);
```

2. Bảng lưu thông tin session (để kiểm soát ongoing)

```
CREATE TABLE SESSION_TRACKING (

    session\_id VARCHAR2(50) PRIMARY KEY,

    user\_id VARCHAR2(50),

    login\_time TIMESTAMP DEFAULT CURRENT\_TIMESTAMP,

    last\_activity\_time TIMESTAMP DEFAULT CURRENT\_TIMESTAMP

);
```

3. Bảng Audit log (ghi nhận vi phạm UCON)

```
CREATE TABLE UCON_AUDIT_LOG (  
  
    log\_id SERIAL PRIMARY KEY,  
  
    user\_id VARCHAR2(50),  
  
    policy\_id VARCHAR2(20),  
  
    event\_time TIMESTAMP DEFAULT CURRENT\_TIMESTAMP,  
  
    description VARCHAR2(255)  
  
);
```

4. PDP – Function đánh giá chính sách (Pseudo-code SQL)

```
CREATE OR REPLACE FUNCTION evaluate_ucon_policy(user_id VARCHAR2, policy_code VARCHAR2)  
  
RETURNS BOOLEAN AS $$  
  
DECLARE  
  
    session\_duration INT;  
  
    rec\_count INT;  
  
BEGIN  
  
    \-- Kiểm tra P1: Session\_time \<= 30 phút  
  
    IF policy\_code \= 'UCON-P1' THEN  
  
        SELECT EXTRACT(EPOCH FROM (CURRENT\_TIMESTAMP \- login\_time)) / 60  
  
        INTO session\_duration  
  
        FROM SESSION\_TRACKING WHERE user\_id \= user\_id;  
  
        IF session\_duration \> 30 THEN
```

```

INSERT INTO UCON\_AUDIT\_LOG(user\_id, policy\_id, description)

VALUES (user\_id, 'UCON-P1', 'Session timeout \- Needs re-auth');

RETURN FALSE; \-- Chặn truy cập

END IF;

END IF;

\-- Kiểm tra P2: Access\_count \<= 50 trong 5 phút

IF policy\_code \= 'UCON-P2' THEN

    SELECT access\_count INTO rec\_count FROM USER\_ATTRIBUTES WHERE user\_id \= user\_id;

    IF rec\_count \> 50 THEN

        UPDATE USER\_ATTRIBUTES SET is\_rate\_limited \= TRUE WHERE user\_id \= user\_id;

        INSERT INTO UCON\_AUDIT\_LOG(user\_id, policy\_id, description)

        VALUES (user\_id, 'UCON-P2', 'Rate limit exceeded');

        RETURN FALSE;

    END IF;

END IF;

RETURN TRUE; \-- Cho phép truy cập

END;

$$ LANGUAGE plpgsql;

```

5. PEP – Trigger kiểm soát trước khi SELECT (PEP = Policy Enforcement Point)

 Cho bảng Medical_Record

CREATE OR REPLACE TRIGGER enforce_ucon_on_select

BEFORE SELECT ON Medical_Record

FOR EACH ROW

DECLARE

 allowed BOOLEAN;

BEGIN

 \-- Kiểm tra session timeout

 allowed := evaluate_ucon_policy(USER, 'UCON-P1');

 IF NOT allowed THEN

 RAISE_APPLICATION_ERROR(-20001, 'Access denied: Session timeout (UCON-P1)');

 END IF;

 \-- Kiểm tra rate limiting

 allowed := evaluate_ucon_policy(USER, 'UCON-P2');

 IF NOT allowed THEN

 RAISE_APPLICATION_ERROR(-20002, 'Access denied: Rate limit exceeded (UCON-P2)');

 END IF;

 \-- Nếu qua kiểm tra, cập nhật thông tin thuộc tính động

 UPDATE USER_ATTRIBUTES

 SET access_count \= access_count \+ 1, last_access_time \= CURRENT_TIMESTAMP

 WHERE user_id \= USER;

END;

✅ **Bạn có thể ghi vào báo cáo:**

"Trigger enforce_ucon_on_select đóng vai trò là **PEP**, mọi lệnh SELECT sẽ bị chặn và chuyển qua **evaluate_ucon_policy()** (đóng vai trò **PDP**) để đánh giá chính sách UCON-P1 và UCON-P2 dựa trên thuộc tính động trong USER_ATTRIBUTES và SESSION_TRACKING (PIP). Cơ chế này minh họa đúng tinh thần UCON: **đánh giá quyền truy cập liên tục và cập nhật thuộc tính sau mỗi hành động.**"

🎁 **Bạn muốn tôi làm tiếp phần nào?**

👉 **Chọn một trong số sau:**

1. **Sinh bảng chính sách UCON_POLICY chuẩn hóa cho báo cáo**
2. **Thiết kế kiến trúc UML / Sơ đồ luồng xử lý "User → PEP → PDP → PIP" dạng đồ họa**
3. **Tạo đoạn văn mô tả "Mô hình triển khai đề xuất" để chèn vào chương 3 khóa luận**
4. **Viết mục "Lợi ích và ý nghĩa khi tích hợp UCON vào RDBMS"**

Bạn muốn đi tiếp hướng nào?

You said:

Hướng dẫn giúp tôi các mục sau: 1.Sinh bảng chính sách UCON_POLICY chuẩn hóa cho báo cáo 2.Thiết kế kiến trúc UML / Sơ đồ luồng xử lý "User → PEP → PDP → PIP" dạng đồ họa 3.Tạo đoạn văn mô tả "Mô hình triển khai đề xuất" để chèn vào chương 3 khóa luận 4.Viết mục "Lợi ích và ý nghĩa khi tích hợp UCON vào RDBMS"

ChatGPT said:

Tuyệt — mình sẽ làm từng phần rõ ràng, sẵn sàng để bạn copy vào báo cáo hoặc dùng để triển khai trực tiếp.

1) Bảng chính sách UCON_POLICY (chuẩn hóa) — DDL (PostgreSQL) + ví dụ chèn

Ghi chú: dưới đây dùng kiểu JSONB cho các điều kiện (để dễ mở rộng). Nếu bạn triển khai trên **Oracle**, thay JSONB bằng CLOB (hoặc VARCHAR2/XMLTYPE tùy nhu cầu) và sửa cú pháp tương ứng.

DDL (PostgreSQL)

-- Bảng chính sách UCON chuẩn (PostgreSQL)

CREATE TABLE ucon_policy (

```

policy\_id      VARCHAR(50) PRIMARY KEY,  \-- mã policy (UCON-P1...)

name            TEXT NOT NULL,          \-- tên ngắn policy

description     TEXT,                   \-- mô tả chi tiết

subject\_expr   TEXT,                   \-- biểu thức chọn subject (ví dụ: role='Doctor')

object\_expr    TEXT,                   \-- biểu thức chọn đối tượng (ví dụ: table='medical\_record')

action         TEXT,                    \-- action (SELECT/UPDATE/DELETE/ALL)

pre\_condition  JSONB,                  \-- điều kiện pre-access (structured JSON)

ongoing\_condition JSONB,              \-- điều kiện ongoing (JSON)

post\_action    JSONB,                  \-- hành động post (ví dụ: giảm trust\_score, log)

effect         VARCHAR(10) NOT NULL,    \-- PERMIT / DENY / REVOKE

priority       INT DEFAULT 100,         \-- ưu tiên policy (số nhỏ \= ưu tiên cao)

enabled        BOOLEAN DEFAULT TRUE,    \-- bật/tắt policy

created\_by     VARCHAR(100),

created\_at     TIMESTAMP WITH TIME ZONE DEFAULT CURRENT\_TIMESTAMP

```

```
);
```

```
-- Ví dụ chèn 2 policy (UCON-P1, UCON-P2)
```

```

INSERT INTO ucon_policy(policy_id, name, description, subject_expr, object_expr, action,

                        pre\_condition, ongoing\_condition, post\_action, effect, priority)

```

```
VALUES
```

```
('UCON-P1',
```

```
'Doctor session timeout',
```

```
'Bác sĩ được SELECT Medical_Record; nếu session > 30 phút => require re-auth hoặc deny ongoing',
```

```

'role = "Doctor"',

'table = "medical_record"',

'SELECT',

'{"trust_score": {"gte": 70}}',

'{"session_duration_minutes": {"lte": 30}, "reauth_on_exceed": true}',

'{"audit": "log_event", "on_violation": {"action": "require_reauth", "severity": "medium"}}',

'PERMIT',

10

),

('UCON-P2',

'Rate limiting: 50 records / 5min',

'Nếu user truy vấn >50 bản ghi trong 5 phút, tạm revoke SELECT và mark rate_limited',

'true', -- áp dụng cho mọi user (hoặc thay bằng role condition)

'table = "medical_record"',

'SELECT',

'{"none": true}',

'{"access_count_last_5min": {"lte": 50}, "on_exceed": {"revoke": true, "duration_minutes": 15}}',

'{"audit": "log_event", "on_violation": {"action": "revoke_temp", "revoke_minutes": 15}}',

'PERMIT',

20

);

```

Giải thích các trường chính

- `subject_expr`, `object_expr`: biểu thức lọc (dạng text) để PDP quyết định policy áp dụng cho subject/object nào — có thể parse thành expression khi đánh giá.

- pre_condition, ongoing_condition, post_action: lưu dưới dạng JSON để dễ mở rộng, ví dụ {"trust_score":{"gte":70}}.
 - effect: có thể là PERMIT, DENY, REVOKE.
 - priority: khi nhiều policy xung đột, PDP xử lý theo priority.
-

2) Thiết kế kiến trúc UML / Sơ đồ luồng xử lý "User → PEP → PDP → PIP"

Mình cung cấp **(A)** mã PlantUML để bạn render trong PlantUML/Live editor và **(B)** một sơ đồ ASCII đơn giản nếu bạn không render được.

(A) PlantUML (copy vào file .puml hoặc trang live PlantUML)

```
@startuml
```

```
title UCON integration: User -> PEP -> PDP -> PIP
```

```
actor "User / Client" as User
```

```
rectangle "RDBMS (Postgres/Oracle)" {
```

```
rectangle "PEP\n(Triggers / DB Hooks)" as PEP
```

```
rectangle "PDP\n(evaluate_policy())" as PDP
```

```
rectangle "PIP\n(user_attrs, session, data_attrs)" as PIP
```

```
database "UCON_POLICY\n(ucon_policy table)" as POLICY
```

```
database "USER_ATTRIBUTES\n(session_tracking, user_attributes)" as ATTR
```

```
database "UCON_AUDIT_LOG" as AUDIT
```

```
}
```

```
User --> PEP : request (SELECT/UPDATE/DELETE/INSERT)
```

```
PEP --> PDP : call evaluate_policy(subject, object, action)
```

```
PDP --> POLICY : read policy definitions
```

```
PDP --> PIP : read attributes (user, session, object)
```

```
PIP --> ATTR : fetch attribute values
```

```
PDP --> PEP : decision (PERMIT / DENY / REVOKE) + obligations
```

PEP --> User : allow / block / require re-auth

PEP --> AUDIT : write event

PDP --> ATTR : post-update attributes (e.g., decrease trust_score)

@enduml

(B) Sơ đồ ASCII ngắn gọn

User/Client

|

v

[PEP - Trigger / DB Hook] <-- chặn mọi request vào table

|

v

[PDP - evaluate_policy()] -- đọc ucon_policy (PIP)

|

/ \

v v

reads attributes (USER_ATTRIBUTES, SESSION_TRACKING) reads policy definitions (UCON_POLICY)

||

v v

decision (PERMIT / DENY / REVOKE / OBLIGATIONS) -----> back to PEP

|

v

PEP enforces decision (allow query / raise error / revoke rights / require re-auth)

|

UCON_AUDIT_LOG (ghi lại, PDP cập nhật attributes nếu cần)

3) Đoạn văn mô tả “Mô hình triển khai đề xuất” — để chèn vào Chương 3

Mô hình triển khai đề xuất

Đề xuất này tích hợp mô hình Usage Control (UCON) trực tiếp vào tầng quản trị cơ sở dữ liệu để đạt khả năng đánh giá quyền truy cập không chỉ ở thời điểm yêu cầu (pre-authorization) mà còn trong suốt quá trình phiên (ongoing control) và sau khi hoàn thành hành động (post-update). Về kiến trúc, trigger và database hook trên RDBMS đóng vai trò **Policy Enforcement Point (PEP)** — chặn mọi yêu cầu truy cập tới các bảng nhạy cảm và chuyển thông tin tới **Policy Decision Point (PDP)**, một bộ hàm thủ tục (ví dụ `evaluate_ucon_policy()`) thực hiện đánh giá chính sách dựa trên định nghĩa lưu trong bảng `ucon_policy`. Các thuộc tính động (Policy Information Point — PIP) như `trust_score`, `access_count`, `session_duration` được lưu trong các bảng `user_attributes` và `session_tracking`. PDP đọc các thuộc tính này, áp dụng logic pre/ongoing/post và trả về quyết định cùng các obligation (ví dụ yêu cầu re-auth, revoke tạm thời). PEP sau đó thi hành quyết định ngay lập tức — có thể từ chối truy vấn, thu hồi quyền trong phiên, hoặc cho phép nhưng kèm yêu cầu bổ sung. Mọi sự kiện vi phạm và quyết định được ghi vào `ucon_audit_log`, và hệ thống sẽ cập nhật thuộc tính động (ví dụ giảm `trust_score`) để ảnh hưởng tới các quyết định sau. Kiến trúc này cho phép kiểm soát liên tục và phản ứng tức thời với hành vi truy cập bất thường, phù hợp cho các hệ thống dữ liệu y tế cần tuân thủ bảo mật cao.

Bạn có thể đặt đoạn văn này vào phần **3.2 Kiến trúc đề xuất** hoặc **3.3 Cơ chế PDP/PEP/PIP**.

4) Mục: "Lợi ích và ý nghĩa khi tích hợp UCON vào RDBMS"

Lợi ích chính

- **Kiểm soát động, theo phiên:** UCON cho phép thay đổi cấp quyền ngay trong phiên truy cập (ongoing control). Điều này giúp ngăn chặn lộ lọt dữ liệu ngay khi phát hiện hành vi bất thường (ví dụ rate-limit, session timeout).
- **Tăng cường an toàn cho dữ liệu nhạy cảm:** Với dữ liệu y tế, các quyết định có thể phụ thuộc vào context (mức độ nhạy cảm của bản ghi, thời điểm, `trust_score`), giảm rủi ro lộ PII/PHI.
- **Ít phụ thuộc công cụ ngoài:** Triển khai trực tiếp trong RDBMS thông qua trigger và stored-procedure tránh cần tích hợp nhiều thành phần bên ngoài, làm đơn giản hoá kiến trúc vận hành và giảm độ trễ giữa phát hiện & thực thi.
- **Khả năng audit và truy vết mạnh:** Mọi quyết định và thay đổi thuộc tính đều được ghi lại trong audit log, thuận lợi cho việc phân tích forensics, báo cáo tuân thủ (compliance).

- **Tự động hóa phản ứng:** Sau vi phạm, hệ thống có thể tự điều chỉnh thuộc tính (ví dụ giảm `trust_score`), tạo vòng feedback tự động giúp bảo vệ lâu dài.

Ý nghĩa học thuật / thực tiễn

- **Nâng cao nghiên cứu về access control:** Việc triển khai UCON trong RDBMS minh hoạ cách mô hình thuộc tính động hoạt động trong thực tế, cung cấp cơ sở thử nghiệm cho so sánh với RBAC/ABAC.
- **Ứng dụng thực tế trong lĩnh vực y tế:** Hệ thống y tế cần kiểm soát ngặt các truy cập vào hồ sơ bệnh nhân; UCON cung cấp cơ chế linh hoạt cho các tình huống khẩn cấp (break-glass), truy cập tạm thời, hoặc điều chỉnh quyền theo lịch sử hành vi.
- **Đóng góp kỹ thuật cho vận hành DBMS:** Bằng cách mô phỏng UCON trên cơ sở trigger + stored-procedure, khóa luận chứng minh khả năng mở rộng các DBMS phổ biến để hỗ trợ kiểu kiểm soát thuộc tính động mà không phải thay đổi lõi hệ thống.

Rủi ro & lưu ý triển khai

- **Hiệu năng:** Kiểm tra ongoing liên tục có thể làm tăng overhead; cần tối ưu PDP (caching policy, giảm số lần gọi) và đánh giá ảnh hưởng thông qua thử nghiệm hiệu năng.
- **Phức tạp chính sách:** Khi sử dụng JSON condition, cần cơ chế quản lý & kiểm thử chính sách chặt chẽ để tránh xung đột hoặc lỗi hổng.
- **Quản lý khóa/đồng bộ:** Việc revoke quyền trong phiên có thể va chạm với transaction đang mở — cần thiết kế rõ ràng hành vi khi revoke (abort transaction / soft deny / delayed revoke).

Kết thúc — đề xuất bước tiếp theo thực thi

Mình đã cung cấp:

- Bảng `ucon_policy` chuẩn + hai ví dụ (UCON-P1, P2).
- Mã PlantUML cho sơ đồ kiến trúc (dễ render).
- Đoạn văn mô tả mô hình (để chèn vào chương 3).
- Mục lợi ích / ý nghĩa.

Bạn muốn mình:

1. Chuyển DDL trên sang **Oracle** (PL/SQL) đầy đủ?
2. Viết **ví dụ trigger + function hoàn chỉnh** cho PostgreSQL (có test data và kịch bản)?
3. Tạo **hình đồ họa PNG/SVG** từ PlantUML và gửi file?

Chọn 1 hoặc kết hợp để mình tiếp tục.

You said:

Trong các gợi ý của bạn về việc tích hợp UCON vào RDBMS, bạn đề nghị dùng PostgreSQL hoặc Oracle. Xin cho tôi biết lý do bạn gợi ý sử dụng 2 hệ quản trị cơ sở dữ liệu này. Nếu tôi chọn tích hợp UCON vào SQL Server hoặc MySQL thì có hạn chế gì không?

ChatGPT said:

Câu hỏi của bạn rất hay và rất “chuẩn đề tài nghiên cứu” 🙌 — vì việc **chọn hệ quản trị CSDL nền tảng để tích hợp UCON** (Usage Control) có ảnh hưởng trực tiếp đến **khả năng hiện thực hóa các thành phần PDP/PEP/PIP**, cũng như **tính động (ongoing)** mà mô hình UCON yêu cầu.

Dưới đây là giải thích chi tiết, chia làm 3 phần:

- 1 Vì sao **PostgreSQL và Oracle** được khuyến nghị.
- 2 Những **hạn chế của SQL Server và MySQL** nếu bạn chọn chúng.
- 3 **Gợi ý hướng xử lý nếu bạn vẫn muốn dùng SQL Server hoặc MySQL.**

1 Vì sao PostgreSQL và Oracle được khuyến nghị

Cả **PostgreSQL** và **Oracle** đều là hệ quản trị **mở rộng tốt và có ngôn ngữ lập trình nội bộ mạnh (procedural language)**, cho phép mô phỏng logic “Usage Control” khá sát với mô hình lý thuyết.

Tiêu chí	PostgreSQL	Oracle
Hỗ trợ PL/SQL / PLpgSQL mạnh	✅ PL/pgSQL gần tương đương PL/SQL, cho phép viết function, trigger, dynamic policy logic .	✅ PL/SQL cực kỳ mạnh, hỗ trợ package, context session, autonomous transaction – dễ quản lý ongoing conditions.
Trigger phong phú (BEFORE/AFTER, FOR EACH ROW/STATEMENT)	✅ Có thể can thiệp sâu vào từng dòng dữ liệu để kiểm tra điều kiện ongoing.	✅ Có đủ loại trigger, thêm cả fine-grained auditing (FGA) .
Policy & Attribute Storage linh hoạt	✅ JSONB, VIEW, hoặc Extension cho ABAC/UCON (như pg_policy hoặc pgaudit).	✅ Có thể tận dụng Virtual Private Database (VPD) và Label Security .
Mở rộng bằng Extension	✅ Dễ viết thêm module mới (C language extension, FDW) để thực hiện PDP riêng.	⚙️ Oracle cho phép package mở rộng, nhưng license thường cao (dùng bản học thuật hoặc trial).

Tiêu chí	PostgreSQL	Oracle
Môi trường học thuật / nghiên cứu	✅ Mã nguồn mở, dễ kiểm thử, ghi log, demo.	⚙️ Có license, nhưng có bản free (Oracle XE).

→ **Tóm lại:**

PostgreSQL = dễ tích hợp, dễ kiểm soát, dễ trình bày kiến trúc UCON.

Oracle = chuẩn công nghiệp, phù hợp với các đặc tính chính thức của UCON (ongoing control, obligation, revocation).

⚙️ 2 Nếu dùng SQL Server hoặc MySQL thì sao?

Tiêu chí	SQL Server	MySQL
Khả năng trigger và procedure	Có T-SQL Trigger , nhưng hạn chế trong việc kiểm soát "ongoing" (không có background job để dàng như PostgreSQL).	Trigger hỗ trợ hạn chế, không có BEFORE SELECT trigger , nên không thể can thiệp trước truy vấn đọc dữ liệu.
Khả năng mô phỏng PDP/PEP	Có thể mô phỏng PEP ở cấp ứng dụng (middleware), nhưng khó nhúng vào engine.	Gần như không thể làm PEP ngay trong DB, chỉ có thể log hoặc chặn DML (INSERT/UPDATE/DELETE).
Ngôn ngữ mở rộng nội bộ	T-SQL mạnh nhưng không mở rộng bằng extension dễ như PostgreSQL.	Không có procedural language mạnh mẽ (PL/pgSQL tương đương).
Hỗ trợ context session / dynamic policy	Có SESSION_CONTEXT() nhưng hạn chế so với Oracle context.	Không có context session thật sự, nên khó xử lý ongoing hoặc obligation.
Khả năng tương thích mô hình UCON	Chỉ có thể mô phỏng một phần pre-authorization và revoke , không xử lý được ongoing control nội tại .	Gần như chỉ làm được phần pre-authorization cơ bản.

🚫 Hạn chế cốt lõi:

- ❌ **Không có trigger cho SELECT** ⇒ không thể kiểm soát quyền truy cập động (ongoing) trên truy vấn đọc.
- ❌ **Không thể cập nhật quyền khi phiên làm việc đang diễn ra** (UCON yêu cầu điều này).
- ❌ **Không có cơ chế mở rộng để gắn PDP/PEP nội tại vào engine.**

3 Gợi ý hướng xử lý nếu vẫn muốn dùng SQL Server hoặc MySQL

Nếu bạn **bắt buộc dùng SQL Server hoặc MySQL**, bạn có thể chọn **kiến trúc phân lớp (middleware)** thay vì tích hợp trực tiếp:

Phương án thay thế:

- **Ứng dụng trung gian (middleware) làm PEP**: intercept truy vấn → gửi yêu cầu đến **PDP API** → PDP đọc dữ liệu chính sách trong DB → trả về **"Allow / Deny / Update attributes"**.
- **CSDL (MySQL/SQL Server)** chỉ đóng vai trò **PIP (Policy Information Point)**, lưu trữ dữ liệu và thuộc tính động.
- Việc cập nhật ongoing hoặc revoke có thể thực hiện thông qua **event trigger hoặc scheduled job**.

Ví dụ:

1. Người dùng gửi truy vấn → Middleware kiểm tra với UCON-PDP API.
2. PDP tra chính sách trong bảng POLICY + thuộc tính user.
3. PDP trả về "allow" hoặc "deny".
4. Nếu "allow", middleware thực hiện truy vấn thật lên SQL Server/MySQL.
5. Nếu "ongoing condition" vi phạm (VD: truy cập quá lâu), PDP cập nhật bảng revoke và middleware chặn truy cập tiếp theo.

→ Cách này **mô phỏng đầy đủ UCON**, chỉ khác là PEP nằm ngoài RDBMS.

Tóm tắt kết luận cho khóa luận

Hệ CSDL	Mức độ phù hợp triển khai UCON	Lý do
PostgreSQL	★★★★	Mã nguồn mở, trigger mạnh, PL/pgSQL linh hoạt, có thể mở rộng PDP/PEP trong engine.
Oracle	★★★★★	PL/SQL mạnh, có Label Security, hỗ trợ ongoing control tự nhiên.
SQL Server	★★	Không hỗ trợ trigger SELECT, PDP phải chạy ngoài DB.
MySQL	★	Hạn chế trigger, khó mô phỏng ongoing condition.

Làm rõ hơn về ngữ cảnh triển khai, cơ sở triển khai và tính khả thi, tính thuyết phục của đề tài sau ""So sánh hiệu năng và độ phức tạp quản lý chính sách giữa RBAC, ABAC và UCON trong một hệ thống thông tin" Mục tiêu: Đánh giá lợi ích và chi phí khi thay thế hoặc kết hợp RBAC/ABAC với UCON. Nội dung chính: Xây dựng môi trường thử nghiệm với 3 mô hình điều khiển truy cập. Đo thời gian xử lý yêu cầu truy cập, độ phức tạp quản trị, và khả năng thích ứng với thay đổi. Đưa ra khuyến nghị áp dụng."

ChatGPT said:

Rất hay — đây là **một đề tài mang tính phân tích – thực nghiệm rất phù hợp cho khóa luận tốt nghiệp CNTT**, đặc biệt trong lĩnh vực **bảo mật hệ thống thông tin và quản lý truy cập**.

Dưới đây là bản hướng dẫn chi tiết giúp bạn **làm rõ ngữ cảnh, cơ sở triển khai, tính khả thi, và tính thuyết phục của đề tài**:

Đề tài:

"So sánh hiệu năng và độ phức tạp quản lý chính sách giữa RBAC, ABAC và UCON trong một hệ thống thông tin"

I. Ngữ cảnh triển khai

Trong các **hệ thống thông tin hiện đại** — ví dụ như hệ thống quản lý bệnh viện, cổng thông tin sinh viên, hoặc nền tảng IoT doanh nghiệp — vấn đề **kiểm soát quyền truy cập (Access Control)** ngày càng quan trọng.

Tuy nhiên, **các mô hình kiểm soát truyền thống (RBAC)** đã bộc lộ hạn chế khi cần xử lý tình huống động như:

- Quyền truy cập thay đổi trong khi phiên làm việc đang diễn ra.
- Chính sách phụ thuộc vào **thuộc tính người dùng, tài nguyên, và ngữ cảnh** (ví dụ: thời gian, vị trí, trạng thái thiết bị).
- Cần kiểm soát điều kiện "đang thực thi" (ongoing conditions) như trong mô hình **UCON (Usage Control)**.

Đề tài này nhằm **đánh giá một cách định lượng và định tính** giữa 3 mô hình:

- **RBAC (Role-Based Access Control)** – quản lý theo vai trò.
- **ABAC (Attribute-Based Access Control)** – quản lý theo thuộc tính.

- **UCON (Usage Control)** – mở rộng ABAC, cho phép thay đổi quyền động trong thời gian thực.

II. Cơ sở triển khai

1. Môi trường thử nghiệm đề xuất

- **Nền tảng RDBMS:** PostgreSQL hoặc Oracle (có hỗ trợ PL/pgSQL hoặc PL/SQL cho trigger và policy).
- **Ngôn ngữ lập trình tích hợp:** Python hoặc Java (dễ tạo mô-đun kiểm thử và đo thời gian).
- **Dữ liệu mô phỏng:**
 - Người dùng (Users)
 - Vai trò hoặc thuộc tính (Roles , Attributes)
 - Tài nguyên (Resources)
 - Bảng ghi log truy cập (Access_Log)

2. Cấu trúc hệ thống mô phỏng

Mỗi mô hình được triển khai như một **module truy cập riêng**:

Mô hình	Cấu trúc chính	Cơ chế quyết định	Đặc trưng
RBAC	Bảng USER_ROLE , ROLE_PERMISSION	So khớp vai trò	Quyền tĩnh, dễ quản lý
ABAC	Bảng USER_ATTRIBUTE , RESOURCE_ATTRIBUTE , POLICY	So khớp thuộc tính	Linh hoạt, đa điều kiện
UCON	Bảng UCON_POLICY , trigger/trigger function	PDP + PEP + PIP	Quyền động, có ongoing condition

Ví dụ mô hình **UCON** kiểm tra:

Cho phép truy cập tài nguyên nếu:

- `User.department = Resource.department`

- `Session.active_time < 15` phút
- `User.license_status = 'valid'`
(và nếu hết hạn → hệ thống tự động thu hồi quyền)

III. Phương pháp so sánh và tiêu chí đánh giá

Tiêu chí	Cách đo lường	Ý nghĩa
Hiệu năng (Performance)	Đo thời gian trung bình xử lý 1.000 yêu cầu truy cập	Cho thấy chi phí xử lý chính sách
Độ phức tạp quản lý (Management Complexity)	Số lượng bảng, số chính sách, số dòng cập nhật khi thay đổi	Đánh giá khả năng mở rộng và bảo trì
Khả năng thích ứng (Adaptability)	Mức độ hỗ trợ thuộc tính động / thay đổi quyền khi phiên đang hoạt động	Phản ánh tính năng động của mô hình
Tính khả thi triển khai (Implementation Cost)	Mức độ khó khi viết trigger/procedure hoặc tích hợp PDP/PEP	Đo chi phí kỹ thuật thực tế

IV. Tính khả thi của đề tài

1. **Phạm vi vừa phải – có thể triển khai trong thời gian khóa luận (~3 tháng):**
 - Mỗi mô hình có thể mô phỏng bằng **5–10 chính sách truy cập**.
 - Dễ triển khai bằng SQL + trigger hoặc Python API.
 - Không cần phần cứng đặc biệt.
2. **Công cụ sẵn có:**
 - PostgreSQL: hỗ trợ **Row-Level Security, trigger function, PL/pgSQL** — lý tưởng để biểu diễn RBAC, ABAC, UCON.
 - Có thể viết script sinh tự động 1.000 yêu cầu truy cập để đo hiệu năng.
3. **Tính thuyết phục học thuật:**
 - So sánh 3 mô hình kinh điển – phù hợp hướng nghiên cứu “Access Control Evolution”.

- Có thể mở rộng bằng việc kết hợp (Hybrid model: RBAC + UCON).
 - Kết quả có thể trực quan bằng biểu đồ (thời gian xử lý, số lần cập nhật, số vi phạm).
-

V. Đề xuất cách thể hiện trong báo cáo

1. Chương 2 – Cơ sở lý thuyết:

- Mô tả chi tiết nguyên lý hoạt động RBAC, ABAC, UCON.
- Bảng so sánh lý thuyết về **Policy Definition, Enforcement, Update Behavior**.

2. Chương 3 – Mô hình triển khai thử nghiệm:

- UML hoặc sơ đồ thành phần (User → PEP → PDP → Database).
- Mỗi mô hình cài đặt module riêng.

3. Chương 4 – Kết quả thực nghiệm:

- Bảng thời gian xử lý 100 / 1.000 / 10.000 yêu cầu truy cập.
- Biểu đồ so sánh “số thao tác quản lý chính sách”.
- Thảo luận kết quả:

RBAC nhanh nhất, nhưng cứng nhắc.

ABAC linh hoạt hơn nhưng phức tạp hơn.

UCON chậm hơn nhưng phù hợp với tình huống yêu cầu điều khiển động.

4. Chương 5 – Kết luận và khuyến nghị:

- Đề xuất kết hợp mô hình (ví dụ: RBAC cho quyền cơ bản + UCON cho điều kiện động).
-

Tính thuyết phục và giá trị đề tài

- **Đóng góp thực tiễn:** cung cấp số liệu rõ ràng giúp tổ chức lựa chọn mô hình điều khiển truy cập phù hợp với nhu cầu.
- **Đóng góp học thuật:** tạo ra bộ dữ liệu và script thử nghiệm có thể tái sử dụng trong nghiên cứu sau.

- **Tính khả thi cao:** dùng phần mềm mã nguồn mở, không yêu cầu hạ tầng lớn, có thể triển khai trên laptop cá nhân.
 - **Có thể mở rộng thành hướng nghiên cứu sau:** “Kết hợp ABAC + UCON cho truy cập động trong môi trường IoT/Cloud”.
-

Nếu bạn muốn, tôi có thể giúp bạn:

1. **Tạo bảng SQL mẫu** cho cả 3 mô hình (RBAC, ABAC, UCON).
2. **Viết sẵn phần “Mô hình triển khai đề xuất”** (chương 3) cho khóa luận này, gồm sơ đồ và mô tả quy trình.

👉 Bạn muốn tôi giúp bạn tiếp theo phần nào: **(1) Cấu trúc bảng mô phỏng 3 mô hình**, hay **(2) Đoạn mô tả chương 3 cho báo cáo?**

Bài báo gốc về UCON:

1. Jaehong Park and Ravi Sandhu: The UCONabc usage control model. ACM Trans. Inf. Syst. Secur., (2004)

<https://profsandhu.com/journals/tissec/p128-park.pdf>